

Predictive Maintenance for Robotic and Automated Machinery: Lightweight Machine-Failure Classification

Learn Depth Academy LLP – Track 1 Capstone, Problem 21 | Author: Pooja Sharma

Abstract

Unplanned failures in robotic work-cells and CNC machines cause costly downtime. This project builds a zero-cost, reproducible machine-failure classifier from five process signals (air and process temperature, rotational speed, torque, tool wear) and machine quality type, using the public AI4I 2020 Predictive Maintenance dataset (10000 records, 3.39% failures). Three foundational models were compared with 5-fold cross-validation. A depth-5 Decision Tree with engineered physical features (temperature difference, mechanical power, wear $\times$ torque) was selected. On the held-out test set it reached recall 0.8529, precision 0.3694, F1 0.5156 and ROC-AUC 0.8825. A Streamlit app validates inputs and returns a failure probability. The dataset is itself a synthetic benchmark, so results indicate feasibility rather than real-world performance.

1. Introduction

Robots, conveyors and CNC cells are the backbone of automated manufacturing. Reactive maintenance stops production, while fixed-interval maintenance wastes component life. Condition-based maintenance uses sensor data to predict failures early. Foundational machine-learning models are attractive because they are cheap to run on edge controllers and easy to explain to maintenance engineers.

2. Problem Definition

Given a single observation of sensor and process readings, predict a binary label: 0 = normal operation, 1 = machine failure in the operating window. Because a missed failure is far costlier than a false alarm, recall on the failure class is the primary metric, with precision and F1 monitored to control alarm fatigue.

3. Related Work

Predictive maintenance has been studied with statistical process control, classical ML (SVM, random forests, gradient boosting) and deep learning on vibration signals. Matzka (2020) released the AI4I 2020 dataset, a synthetic benchmark of a milling machine with five failure modes, to support explainable predictive-maintenance research. This work follows that schema but restricts itself to interpretable foundational models. (References should be verified before submission.)

4. Dataset

The AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020; UCI Machine Learning

Repository, CC BY 4.0) contains 10000 records of a simulated milling machine. Columns: product ID, quality type (L/M/H), air temperature [K], process temperature [K], rotational speed [rpm], torque [Nm], tool wear [min], the binary target Machine failure, and five failure-mode flags (TWF tool wear, HDF heat dissipation, PWF power, OSF overstrain, RNF random). The copy used here was obtained from a public GitHub repository mirroring the original file (rahul4018/Predictive-Maintenance-AI4I2020) and verified against the published statistics (10,000 rows, 339 failures). The data are synthetic but designed to reflect real predictive-maintenance behaviour. Data-quality check: 0 duplicate rows and no missing values were found.

5. Methodology

Duplicate rows were dropped (none found) and a median imputer inside the pipeline guards against missing values without leakage. The five failure-mode columns (TWF, HDF, PWF, OSF, RNF) were excluded from the features because they are direct components of the target; using them would give artificially perfect scores. Three engineered features encode the failure physics: temperature difference, mechanical power $P = \text{torque} \times 2\pi \cdot \text{rpm}/60$, and wear $\times$ torque. Numeric features were standardised and quality type one-hot encoded. An 80/20 stratified split (seed 42) was used, with 5-fold stratified CV on the training portion for model selection.

6. Exploratory Data Analysis

The class balance is 9661 normal to 339 failure records (3.39%), a strong imbalance. Failure modes: HDF 115, OSF 98, PWF 95, TWF 46, RNF 19. Individual features overlap strongly between classes, which is why linear models on raw features perform poorly. Box plots show speed and torque outliers; they were kept because extreme values correspond to genuine failure conditions (power and overstrain failures).



Fig. 1 – Class balance and failure modes

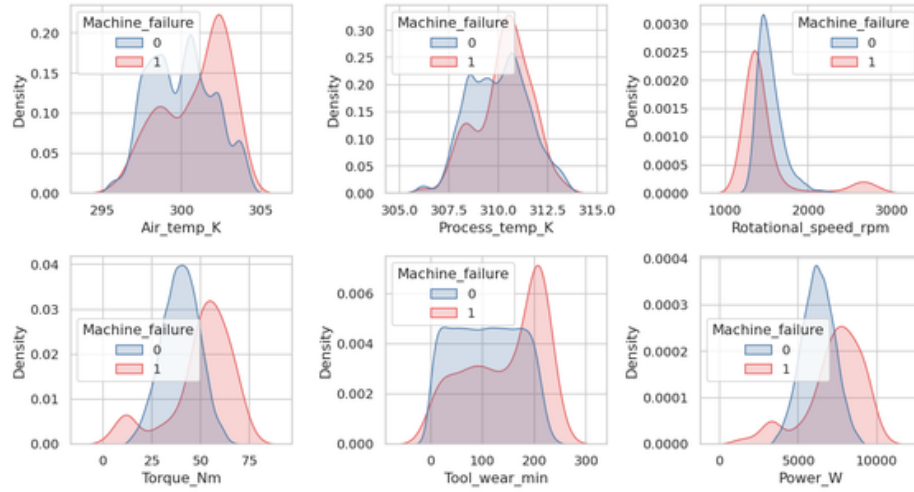


Fig. 2 – Feature distributions by class

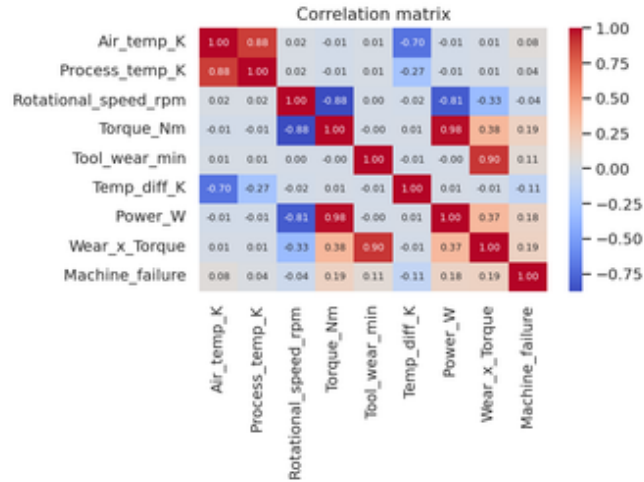


Fig. 3 – Correlation matrix

7. Model Development

Logistic Regression (balanced class weights), KNN (k=7) and a Decision Tree (max depth 5, balanced weights) were compared. Cross-validation results are shown in Table 1.

Model	Recall	Precision	F1	ROC-AUC
LogReg (raw feats)	0.804	0.137	0.235	0.897

LogReg (engineered)	0.841	0.162	0.271	0.923
KNN (engineered)	0.273	0.786	0.405	0.881
DecisionTree d=5 (engineered)	0.923	0.41	0.568	0.94

Table 1 – 5-fold CV results on training data

The Decision Tree gives the best F1 and recall. Failure rules are threshold-based and non-linear, which trees represent naturally, whereas a linear model cannot express a power band, and KNN suffered in the imbalanced, scaled space (high precision, low recall). Feature engineering lifted Logistic Regression recall from 0.804 to 0.841 and ROC-AUC from 0.897 to 0.923, confirming the value of domain features.

8. Results

Test set (n=2000): accuracy 0.9455, precision 0.3694, recall 0.8529, F1 0.5156, ROC-AUC 0.8825.

Confusion matrix: TN=1833, FP=99, FN=10, TP=58. A majority-class classifier would score 0.966 accuracy, so accuracy is not informative here; the model catches 85.3% of failures while raising about 99 false alarms in 1932 normal cases.

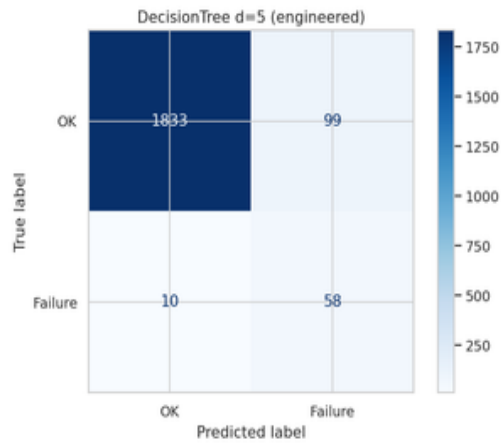


Fig. 4 – Confusion matrix (test)

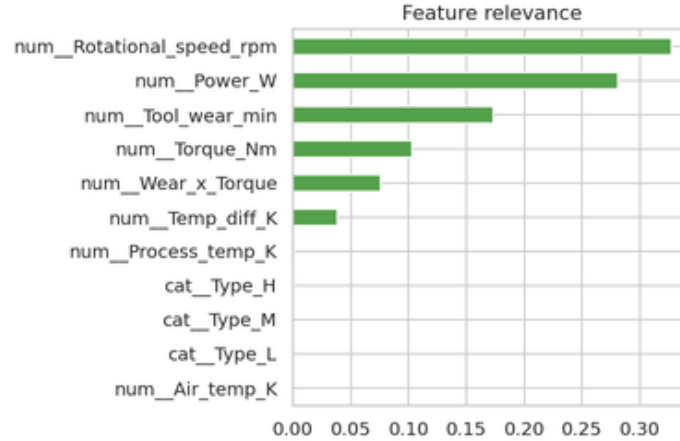


Fig. 5 – Feature relevance

9. Discussion

Rotational speed (importance 0.327) and mechanical power (0.281) are the most relevant features, followed by tool wear (0.173) and torque (0.103); this matches the failure mechanisms (heat dissipation and power failures depend on speed, torque and their product). Error analysis: of the 10 missed failures, 3 were tool-wear, 2 heat-dissipation and 3 overstrain failures and none were power failures; tool-wear failures are partly random by construction, so they are hard to predict from the sensors. The 99 false positives are normal observations close to failure thresholds (mean power 6191 W). Precision is low (0.3694) because failures are only 3.4% of the data, so most alarms are false; in maintenance this is often acceptable given the recall, and the decision threshold can be tuned to the cost of a missed failure versus an unnecessary inspection. The gap between CV ROC-AUC (0.940) and test ROC-AUC (0.8825) reflects the small number of test failures (68) and the coarse probabilities of a shallow tree.

10. Limitations

The dataset is synthetic and its failure rules are deterministic thresholds, so a tree can recover them more easily than it could real machine data with noise, drift and time dependence. Only 68 failures are in the test split, so the metrics have wide uncertainty. Observations are treated as independent rather than as time series, there is no calibration analysis, and a single train/test split was used for the final estimate.

11. Future Scope

Validate on real robot/CNC sensor logs (e.g., public run-to-failure datasets); add rolling-window and vibration features; calibrate probabilities and tune the threshold by cost; predict the failure mode (multi-label); deploy on an edge controller or ROS node

subscribing to joint-sensor topics.

12. Conclusion

A small, interpretable Decision Tree with physics-inspired features detects 85.3% of machine failures in the AI4I 2020 benchmark and is served through a validated Streamlit interface. The project demonstrates a full reproducible ML workflow and highlights why recall, leakage control and feature engineering matter in predictive maintenance for automation.

References

- [1] S. Matzka, “Explainable Artificial Intelligence for Predictive Maintenance Applications,” Proc. 3rd Int. Conf. on Artificial Intelligence for Industries (AI4I), 2020. Dataset: S. Matzka, AI4I 2020 Predictive Maintenance Dataset, UCI Machine Learning Repository, <https://doi.org/10.24432/C5HS5C>, CC BY 4.0. Mirror used: <https://github.com/rahul4018/Predictive-Maintenance-AI4I2020>.
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [3] Streamlit, open-source app framework, <https://streamlit.io>. pandas, NumPy, Matplotlib and seaborn documentation.